

分治法

《算法设计与分析》

信息科学与技术学院 曾华琳



01

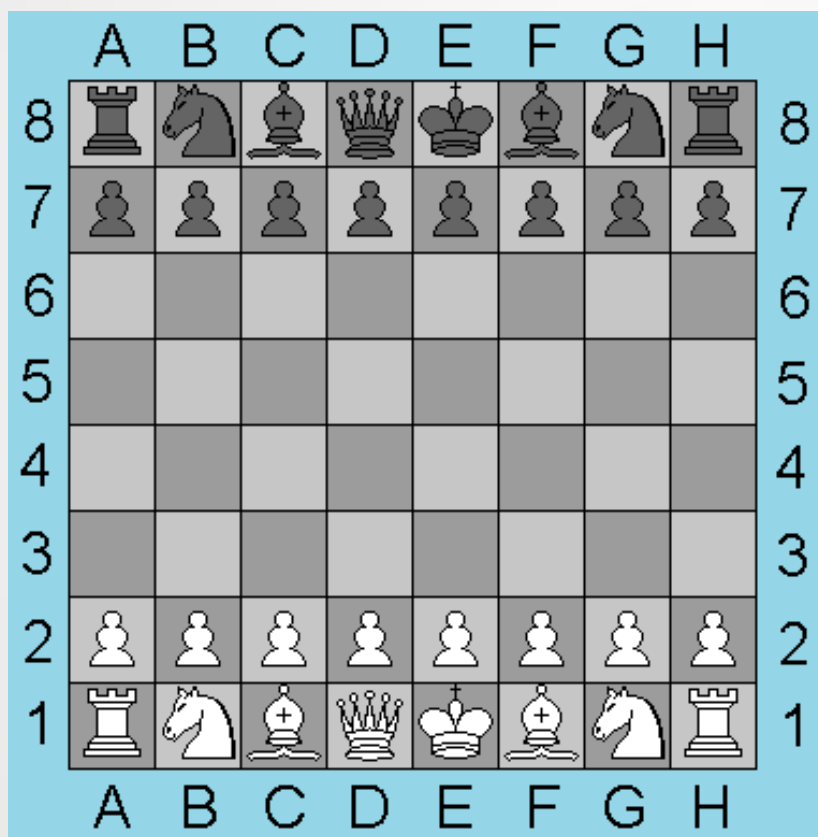


覆盖残缺棋盘



覆盖残缺棋盘

A real chessboard





覆盖残缺棋盘

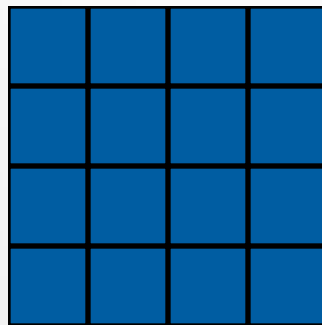
A 是一个 $n \times n$ 的棋盘， n 为2的整数幂。



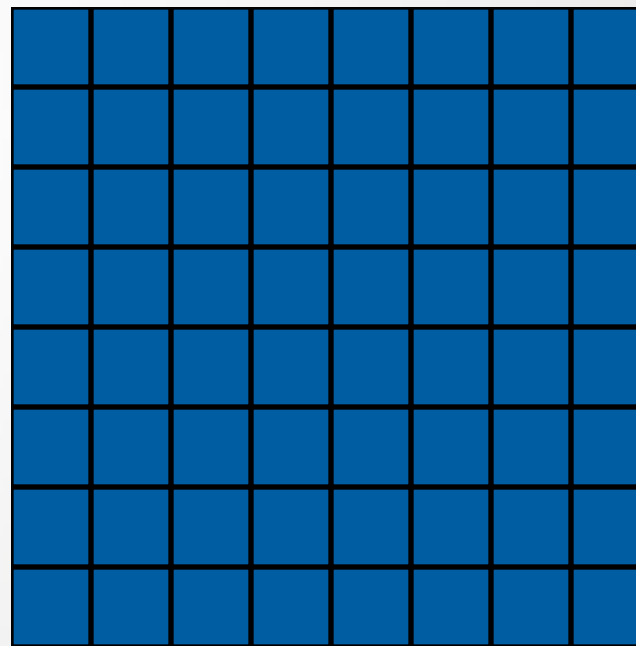
1×1



2×2



4×4



8×8



覆盖残缺棋盘

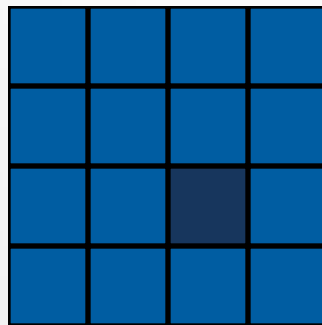
残缺棋盘



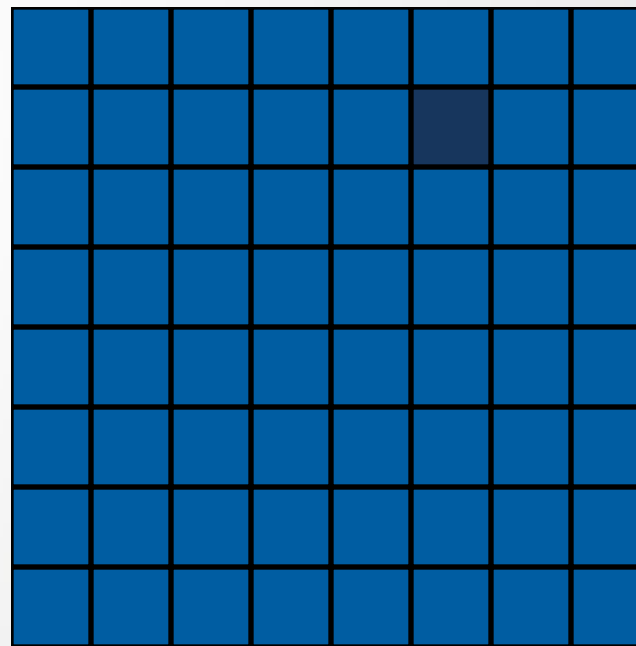
1×1



2×2



4×4



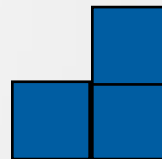
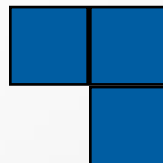
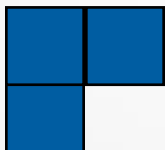
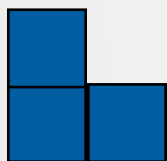
8×8



三格板

三格板是一种 L 形的物体，它能覆盖棋盘的三个格子

三格板有四个方向



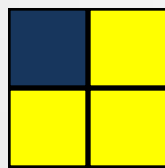


三格板

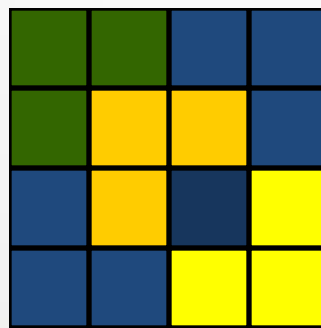
- 用 $(n^2 - 1)/3$ 个三重格放置在 $n \times n$ 的缺陷棋盘上，正好能够覆盖所有方格。



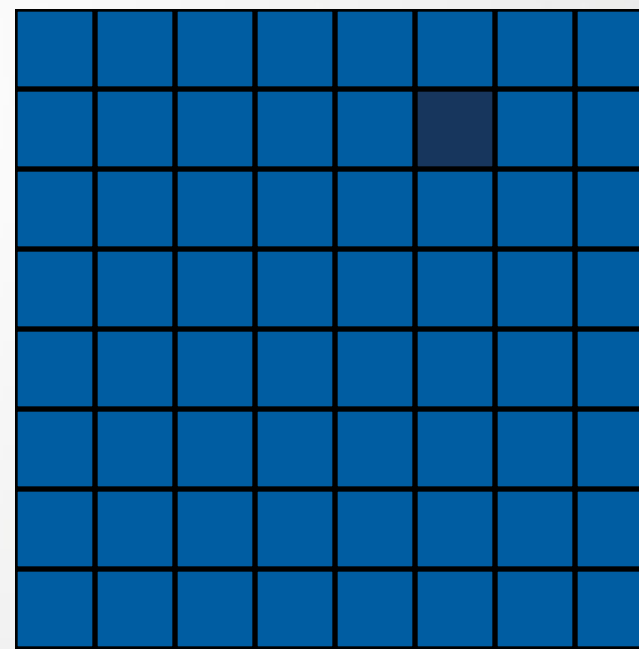
1 × 1



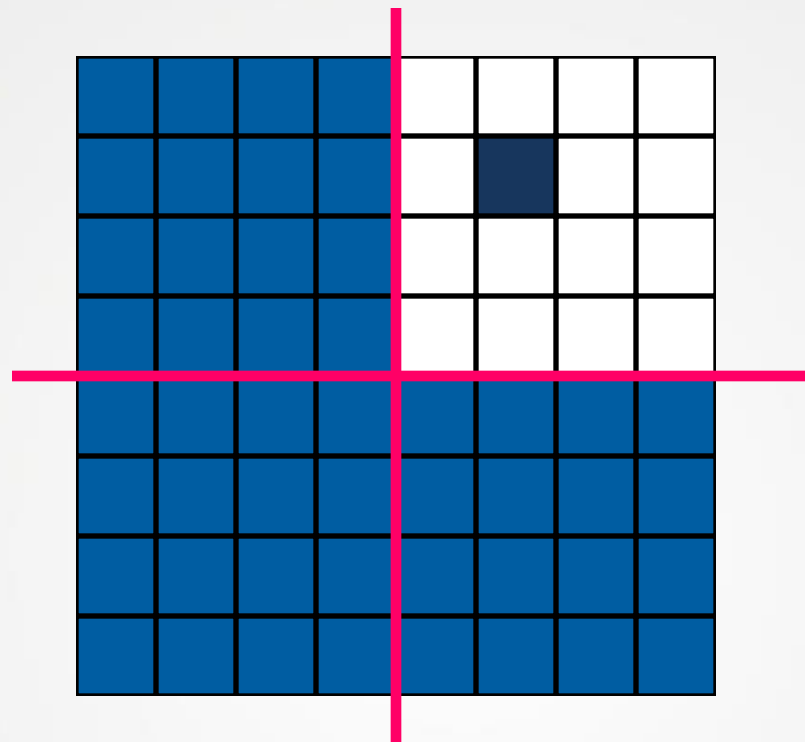
2 × 2



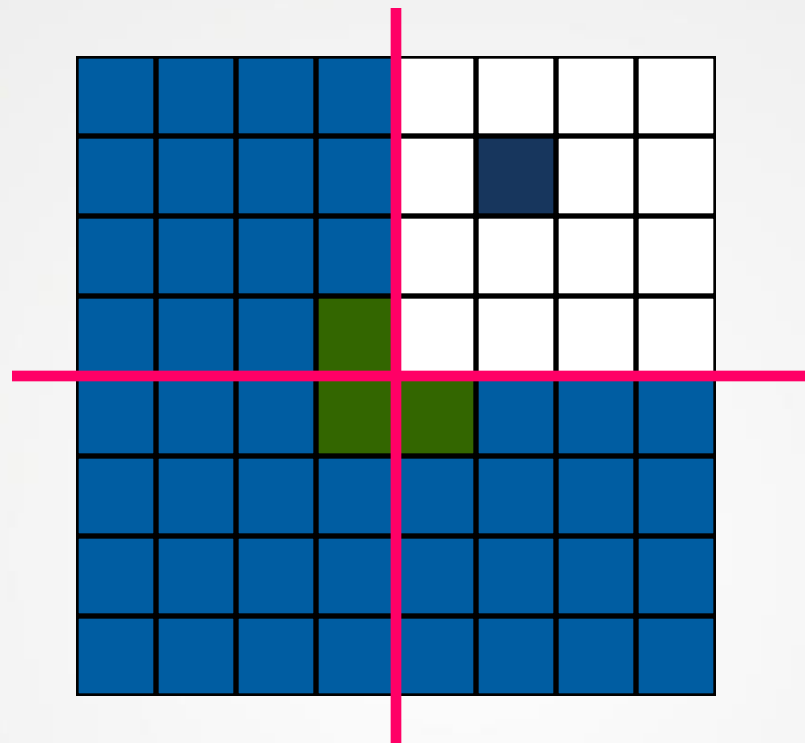
4 × 4



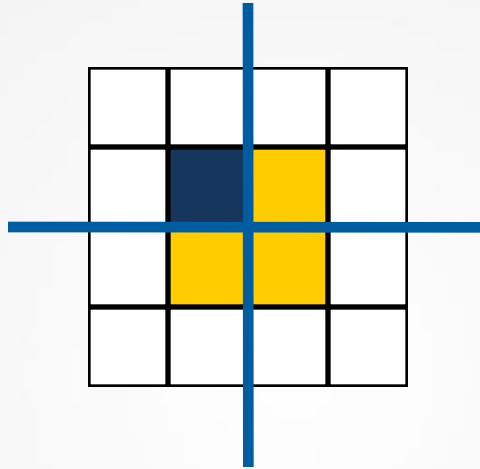
8 × 8



- 划分为四个小棋盘。
- 其中一个是一个4×4缺陷棋盘。



- 在其它三个 4×4 棋盘都相邻的拐角上放一个三格板，使它们也成为缺陷棋盘。
- 递归地覆盖四个 4×4 缺陷棋盘。





覆盖残缺棋盘

```
void chessBoard(int tr, int tc, int dr, int dc, int size)
{
    board[tr + s - 1][tc + s] = t;
    if (size == 1) return;           // 覆盖其余方格
    int t = tile++; // L型骨牌号      chessBoard(tr, tc+s, tr+s-1, tc+s, s);}
    s = size/2; // 分割棋盘           // 覆盖左下角子棋盘
}
```

复杂度分析

$$T(k) = \begin{cases} O(1) & k = 0 \\ 4T(k-1) + O(1) & k > 0 \end{cases}$$

$T(n)=O(4^k)$ 渐进意义下的最优算法

```
// 覆盖右上角子棋盘
if (dr < tr + s && dc >= tc + s)
    // 特殊方格在此棋盘中
    chessBoard(tr, tc+s, dr, dc, s);
else { // 此棋盘中无特殊方格
    // 用 t 号L型骨牌覆盖左下角
    // 特殊方格在此棋盘中
    chessBoard(tr+s, tc+s, dr, dc, s);
else { // 用 t 号L型骨牌覆盖左上角
    board[tr + s][tc + s] = t;
    // 覆盖其余方格
    chessBoard(tr+s, tc+s, tr+s, tc+s, s);}
}
```

谢谢大家

